



universität
uulm

Interactive Computer Graphics

Chapter 3 – Geometric Transformations

Summer Term 2026

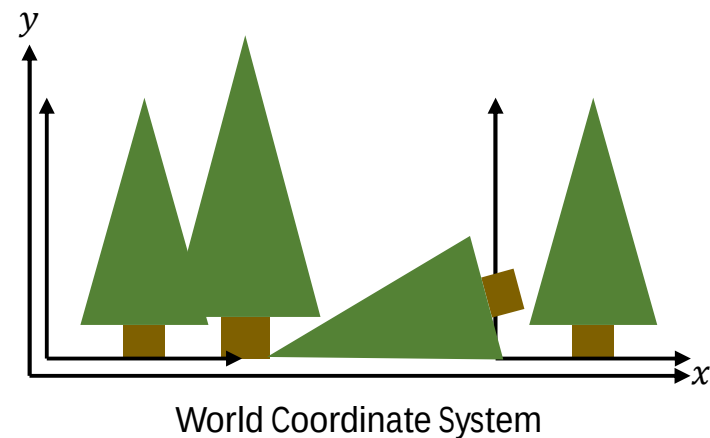
Timo Ropinski



Visual Computing Group
Institute of Media Informatics

Transforming 3D Models

- 3D models need to be positioned and oriented in the virtual world (sometimes scaling and other modifications are necessary)
- These modifications are performed by applying transformations to the vertices describing a 3D model
- Transformations are applied through matrix vector multiplication



Chapter Outline

3.1 Mathematical Foundations

3.2 Affine Transformations

3.3 Composition of Transformations

3.4 Coordinate System Change

3.5 Transformations in OpenGL

References



universität
uulm

3.1 Mathematical Foundations

Vector and Matrix Calculus



Visual Computing Group
Institute of Media Informatics

Vertices as Vectors

- Vectors are used to represent geometric objects (coordinates)
- Vectors have real numbers as components, so they are elements of $\mathbb{R}^2, \mathbb{R}^3, \mathbb{R}^4$
 - In computer graphics mostly $\mathbb{R}^3$ or $\mathbb{R}^4$ vectors used
- Vertices of a polygon are represented as column vectors

$$\vec{v} = \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix}, \vec{v} = \begin{pmatrix} v_x \\ v_y \\ v_z \\ v_w \end{pmatrix}$$

Transformations as Matrices

- Matrices are used to represent transformations
- In computer graphics mainly 4×4 square matrices are used

$$M = \begin{pmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{pmatrix}$$

- Unit matrix I : all $A_{ij} = 0$ ($i \neq j$) and all $A_{ii} = 1$ (main diagonal)
- Inverse matrix M^{-1} (for square matrices) : $M^{-1} \cdot M = M \cdot M^{-1} = I$

Matrix Vector Multiplication

- Since vectors describe vertices and matrices describe transformations, transformations can be applied through matrix vector multiplication
- Input: $(m \times n)$ matrix A and $\mathbb{R}^m$ vector B
- Result: $\mathbb{R}^n$ vector C , whereby B is transformed through A
- Example for (4×4) matrix A and $\mathbb{R}^4$ vector B :

$$\begin{pmatrix} a & b & c & d \\ e & f & g & h \\ i & j & k & l \\ m & n & o & p \end{pmatrix} \cdot \begin{pmatrix} x \\ y \\ z \\ w \end{pmatrix} = \begin{pmatrix} a \cdot x + b \cdot y + c \cdot z + d \cdot w \\ e \cdot x + f \cdot y + g \cdot z + h \cdot w \\ i \cdot x + j \cdot y + k \cdot z + l \cdot w \\ m \cdot x + n \cdot y + o \cdot z + p \cdot w \end{pmatrix}$$

- General multiplication rule is $c_{j,i} = \sum_{k=1}^m a_{k,i} \cdot b_{j,k}$



universität
uulm

3.2 Affine Transformations

Translation, Rotation, Scaling, and more



Visual Computing Group
Institute of Media Informatics

Translations – Moving 3D Models

- Matrix representation of a translation

$$T(d_x, d_y, d_z) = \begin{pmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Application of a translation

$$T(d_x, d_y, d_z) \cdot v = \begin{pmatrix} 1 & 0 & 0 & d_x \\ 0 & 1 & 0 & d_y \\ 0 & 0 & 1 & d_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \\ v_z \\ 1 \end{pmatrix} = \begin{pmatrix} v_x + d_x \\ v_y + d_y \\ v_z + d_z \\ 1 \end{pmatrix}$$

Translation Properties

- Lengths and angles are preserved
- Identity: $T(0, 0, 0) = I$ (identity matrix)
- Changing order of translations:
 - $T(d1_x, d1_y, d1_z) \cdot T(d2_x, d2_y, d2_z) = T(d2_x, d2_y, d2_z) \cdot T(d1_x, d1_y, d1_z)$
- Concatenating translations:
 - $T(d1_x, d1_y, d1_z) \cdot T(d2_x, d2_y, d2_z) = T(d1_x + d2_x, d1_y + d2_y, d1_z + d2_z)$
- Inverting translations:
 - $T^{-1}(d_x, d_y, d_z) = T(-d_x, -d_y, -d_z)$

Scaling – Resizing 3D Models

- Matrix representation of a scaling

$$S(s_x, s_y, s_z) = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Application of a scaling

$$S(s_x, s_y, s_z) \cdot v = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \\ v_z \\ 1 \end{pmatrix} = \begin{pmatrix} v_x \cdot s_x \\ v_y \cdot s_y \\ v_z \cdot s_z \\ 1 \end{pmatrix}$$

Scaling Properties

- Lengths are not preserved, angles only for uniform scalings
- Identity: $S(1, 1, 1) = I$ (identity matrix)
- Changing order of scalings:
 - $S(s1_x, s1_y, s1_z) \cdot S(s2_x, s2_y, s2_z) = S(s2_x, s2_y, s2_z) \cdot S(s1_x, s1_y, s1_z)$
- Concatenating scalings:
 - $S(s1_x, s1_y, s1_z) \cdot S(s2_x, s2_y, s2_z) = S(s1_x \cdot s2_x, s1_y \cdot s2_y, s1_z \cdot s2_z)$
- Inverting scalings:
 - $S^{-1}(s_x, s_y, s_z) = S\left(\frac{1}{s_x}, \frac{1}{s_y}, \frac{1}{s_z}\right)$

Rotation – Changing Orientation of 3D Models 1/2

- Matrix representation of a rotation around x-, y- and z-axis

$$R_x(\theta) = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_y(\theta) = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

$$R_z(\theta) = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation – Changing Orientation of 3D Models 2/2

- Rotations are made in the right-handed coordinate system so that when we look along a positive axis to the origin, a 90° rotation about that axis interleaves the positive axes as follows
 - Rotation around x-axis transforms $+y$ to $+z$
 - Rotation around y-axis transforms $+z$ to $+x$
 - Rotation around z-axis transforms $+x$ to $+y$

Rotation Properties

- Lengths and angles are preserved
- Identity: $R_x(0) = R_y(0) = R_z(0) = I$ (identity matrix)
- Changing order of rotations around same axis:
 - $R_a(\theta) \cdot R_a(\varphi) = R_a(\varphi) \cdot R_a(\theta)$
- Concatenating rotations around same axis:
 - $R_a(\theta) \cdot R_a(\varphi) = R_a(\theta + \varphi)$
- Inverting rotations:
 - $R^{-1}(\theta) = R(-\theta) = R^T(\theta)$

- Rotations around different axis are not commutative!

Mirroring – Reflecting 3D Models on a Plane

- Matrix representation of mirroring along x-y plane

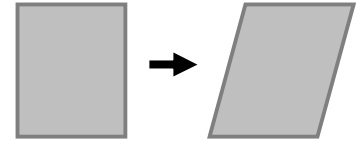
$$M_z = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Application of mirroring

$$M_z \cdot v = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \\ v_z \\ 1 \end{pmatrix} = \begin{pmatrix} v_x \\ v_y \\ -v_z \\ 1 \end{pmatrix}$$

- Mirroring converts between left- and right-handed coordinate system
- Mirroring preserves lengths and angles, but not the polygon orientation (!)

Shearing – Skewing 3D Models



- Matrix representation of a shearing

$$SH(h_{xy}, h_{xz}, h_{yx}, h_{yz}, h_{zx}, h_{zy}) = \begin{pmatrix} 1 & h_{xy} & h_{xz} & 0 \\ h_{yx} & 1 & h_{yz} & 0 \\ h_{zx} & h_{zy} & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

- Application of shearing

$$SH_{xy}(a) \cdot v = \begin{pmatrix} 1 & a & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} v_x \\ v_y \\ v_z \\ 1 \end{pmatrix} = \begin{pmatrix} v_x + av_y \\ v_y \\ v_z \\ 1 \end{pmatrix}$$



universität
uulm

3.3 Composition of Transformations

Concatenation of several transformation steps



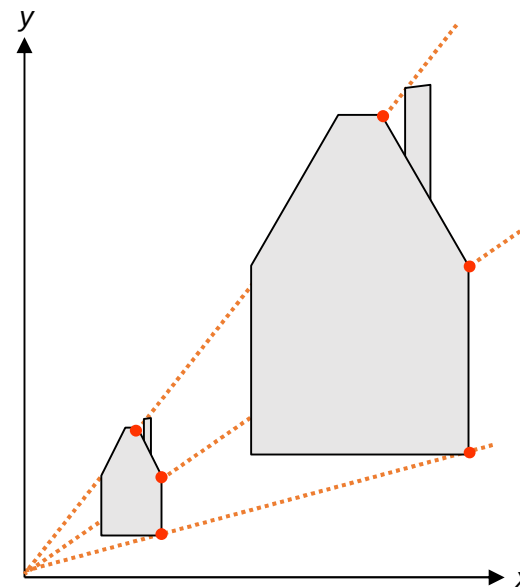
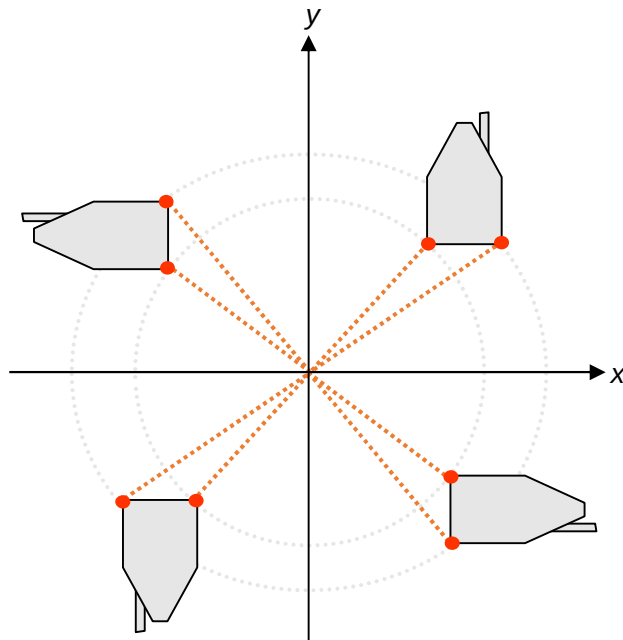
Visual Computing Group
Institute of Media Informatics

Composition of Transformations

- Composition of transformations T_i is done by multiplying the respective transformation matrices
- $(T_1 \cdot T_2 \cdot \dots \cdot T_{n-1} \cdot T_n) \cdot v = (T_1 \cdot (T_2 \cdot (\dots \cdot (T_{n-1} \cdot (T_n \cdot v))))$
 - Interpretation: apply first transformation T_n , then T_{n-1} , ..., and finally T_1
- Application of composited transformation is more efficient!
- Reminder: matrix multiplication generalizes the vector-matrix product
 - Result entries are calculated as follows
- $c_{j,i} = \sum_{k=1}^m a_{k,i} \cdot b_{j,k}$

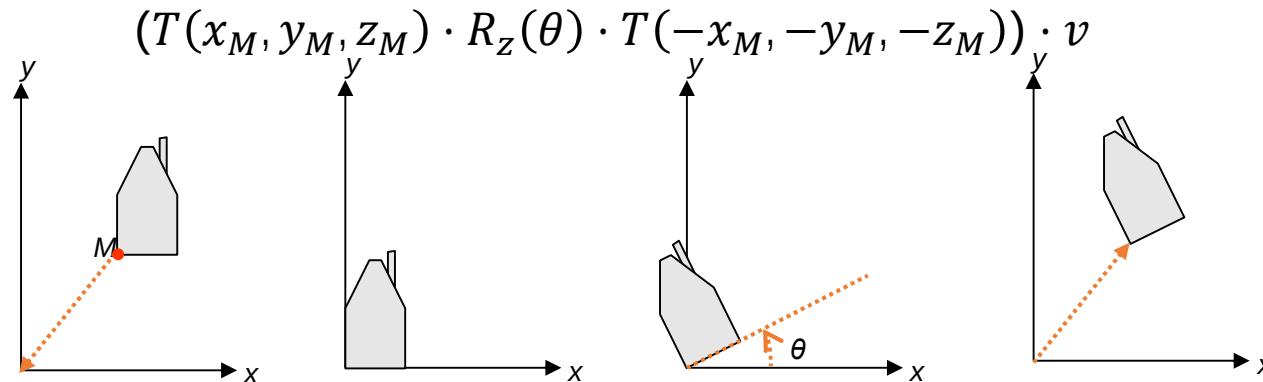
Reference Points 1/2

- Rotation and scaling happen in relation to origin (reference point)



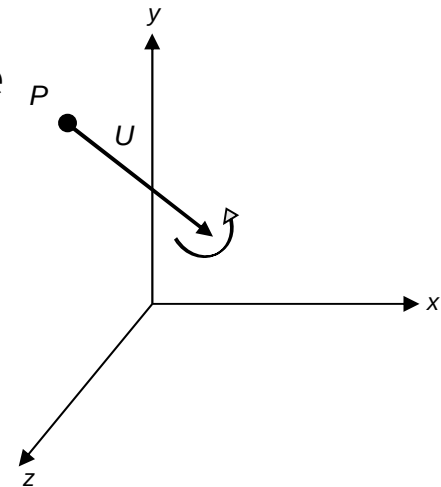
Reference Points 2/2

- In order to rotate (or scale) 3D models at an arbitrary reference point, a sequence of transformations is necessary
 - Move the 3D model from reference point M to the origin O
 - Rotate (or scale) the 3D model
 - Move the 3D model back to M
- Example: Rotation of θ around $M = (x_M, y_M, z_M)$



Rotation Around Arbitrary Axis

- Rotation around any axis
 - Axis given by center of rotation P and direction $U = (u_x, u_y, u_z)$
 - Rotation by θ degrees
- Construction of transformation as transformation sequence
 - Move center of rotation P to the origin
 - Rotation around the y-axis so that U lies in the yz-plane
 - Rotation about the x-axis so that U lies on the z-axis
 - Rotation about the z-axis by θ degrees
 - Reverse rotation about the x-axis
 - Reverse rotation about the y-axis
 - Backward shift to P



Reminder: Vector Calculus

- Length of a vector is expressed as follows

$$|\vec{v}| = \sqrt{v_x^2 + v_y^2 + v_z^2}$$

- Vectors of length $|\vec{v}| = 1$ are normalized and called unit vectors
- Arbitrary vectors can be normalized as follows

$$\vec{n} = \frac{1}{|\vec{v}|} \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix}$$

Reminder: Cross Product (= vector product)

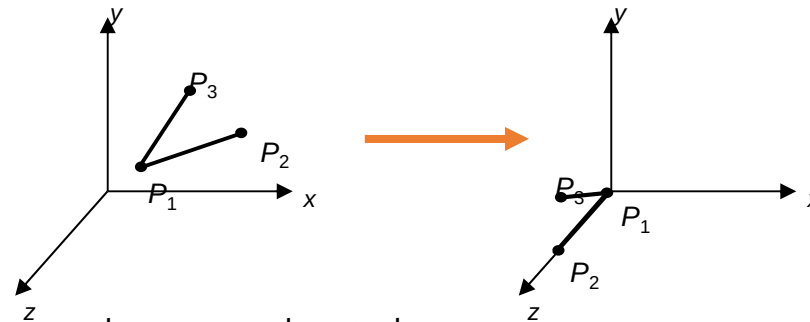
- Given two vectors $\vec{u}$ and $\vec{v}$ in $\mathbb{R}^3$
- Cross product $\vec{u} \times \vec{v}$ of $\vec{u}$ and $\vec{v}$ is a vector defined as follows

$$\vec{u} \times \vec{v} = \begin{pmatrix} u_x \\ u_y \\ u_z \end{pmatrix} \times \begin{pmatrix} v_x \\ v_y \\ v_z \end{pmatrix} = \begin{pmatrix} u_y \cdot v_z - u_z \cdot v_y \\ u_z \cdot v_x - u_x \cdot v_z \\ u_x \cdot v_y - u_y \cdot v_x \end{pmatrix}$$

- Geometric interpretation
 - $\vec{u} \times \vec{v}$ is perpendicular to the plane defined by $\vec{u}$ and $\vec{v}$
 - Length of $\vec{u} \times \vec{v}$ is equal to the parallelogram area spanned by $\vec{u}$ and $\vec{v}$

Example – Transforming Directed Line Segment 1/7

- Example transformation task
 - Given line segments P_1P_2 and P_1P_3
 - Transform it into the yz-plane so that P_1P_2 lies on the z-axis
 - Length and angles shall be preserved



- Two alternative approaches can be taken
 - Determine rotation angles for rotation sequence
 - Exploit properties of orthogonal matrices

Example – Transforming Directed Line Segment 2/7

- Approach 1: Determine rotation angles for rotation sequence
 - Move $P_1 = (x_1, y_1, z_1)$ into the origin
 - Rotate around the y-axis so that P_1P_2 lies in the yz-plane
 - Rotate around the x-axis so that P_1P_2 lies on the z-axis
 - Rotate around the z-axis so that P_1P_3 lies in the yz-plane
 - Step 1 – Move P_1 into origin
 - Translation $T(-x_1, -y_1, -z_1)$
 - New coordinates
 - $P'_1 = (0,0,0)$
 - $P'_2 = (x_2 - x_1, y_2 - y_1, z_2 - z_1)$
 - $P'_3 = (x_3 - x_1, y_3 - y_1, z_3 - z_1)$

Example – Transforming Directed Line Segment 3/7

- Step 2 – Rotation of $P'_1P'_2$ around y-axis into yz-plane

- Rotation $R_y(\theta-90^\circ)$

- Rotation angle $-(90^\circ-\theta) = \theta-90^\circ$

- Matrix ingredients

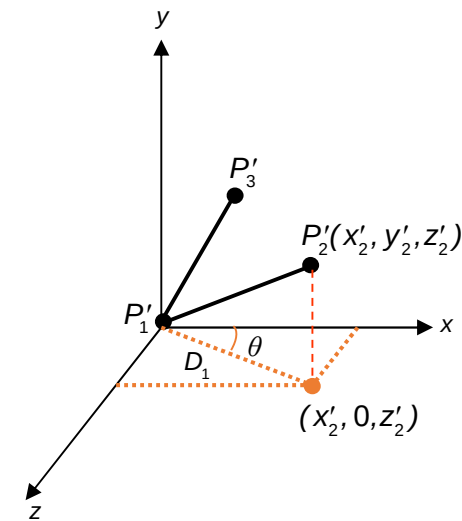
- $\cos(\theta-90^\circ) = \sin(\theta) = \frac{z'_2}{D_1} = \frac{z_2-z_1}{D_1}$

- $\sin(\theta-90^\circ) = -\cos(\theta) = -\frac{x'_2}{D_1} = -\frac{x_2-x_1}{D_1}$

- $D_1 = \sqrt{(x'_2)^2 + (z'_2)^2}$

- New coordinates

- $P''_i = R_y(\theta-90^\circ) \cdot P'_i$



Example – Transforming Directed Line Segment 4/7

- Step 3 – Rotation of $P''_1P''_2$ around x-axis onto z-axis

- Rotation $R_x(\varphi)$
- Matrix ingredients

- $\cos(\varphi) = \frac{z''_2}{D_2}$

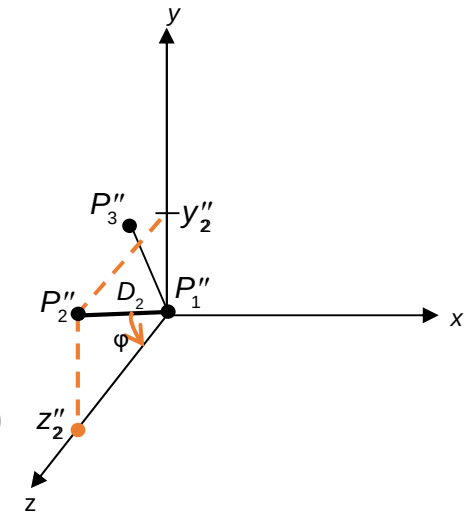
- $\sin(\varphi) = \frac{y''_2}{D_2}$

- $D_2 = |P''_1P''_2| = |P_1P_2|$

- New coordinates

- $P'''_i = R_x(\varphi) \cdot P''_i = R_x(\varphi) \cdot R_y(\theta - 90^\circ) \cdot P'_i$

- $P'''_2 = R_x(\varphi) \cdot P''_2 = R_x(\varphi) \cdot R_y(\theta - 90^\circ) \cdot P'_2 = (0, 0, |P_1P_2|)$

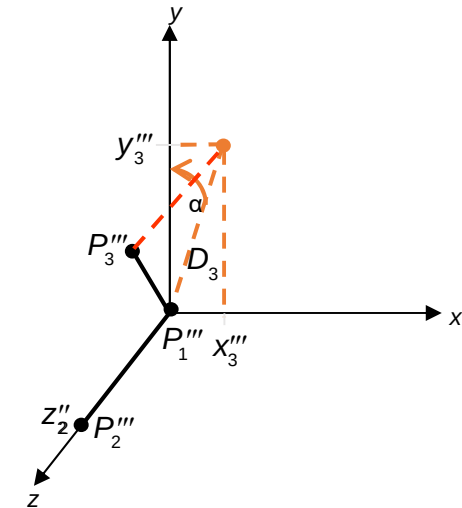


Example – Transforming Directed Line Segment 5/7

- Step 4 – Rotation of $P'''_1P'''_3$ into yz-plane

- Rotation $R_z(\alpha)$
- Matrix ingredients

- $\cos(\alpha) = \frac{y'''_3}{D_3}$
- $\sin(\alpha) = \frac{x'''_3}{D_3}$
- $D_3 = \sqrt{(x'''_3)^2 + (y'''_3)^2}$



- Entire transformation M is a rigid body transformation which preserves lengths and angle

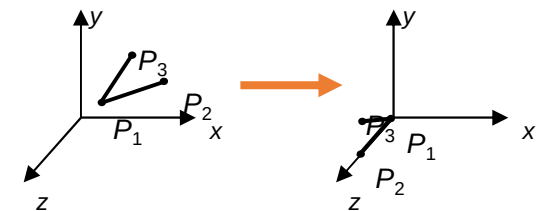
$$M = R_z(\alpha) \cdot R_x(\varphi) \cdot R_y(\theta - 90^\circ) \cdot T(-x_1, -y_1, -z_1)$$

Example – Transforming Directed Line Segment 6/7

- Approach 2: Construct orthogonal matrix which obeys following properties
 - Each row and column corresponds to a unit length vector
 - Row and column vectors are orthogonal to each other
 - Each row vector rotates on one main axis
(R_x on x-axis, R_y on y-axis, R_z on z-axis)

- Orthogonal matrix notation

$$\bullet R = \begin{pmatrix} r_{1x} & r_{2x} & r_{3x} \\ r_{1y} & r_{2y} & r_{3y} \\ r_{1z} & r_{2z} & r_{3z} \end{pmatrix} = \begin{pmatrix} R_x \\ R_y \\ R_z \end{pmatrix} = (R_1 \quad R_2 \quad R_3)$$



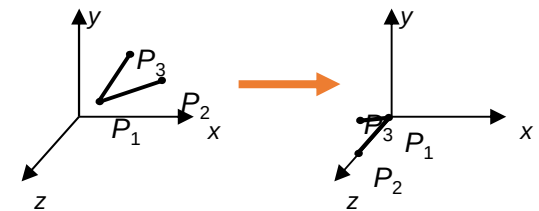
- Consideration 1: Formulate R_z as unit vector rotating on z-axis

$$\bullet R_z = (r_{1z} \quad r_{2z} \quad r_{3z})^T = \frac{P_1 P_2}{|P_1 P_2|}$$

Example – Transforming Directed Line Segment 7/7

- Consideration 2: Formulate R_x as unit vector orthogonal to the plane spanned by P_1 , P_2 , and P_3
 - $R_x = (r_{1x} \quad r_{2x} \quad r_{3x})^T = \frac{P_1 P_3 \times P_1 P_2}{|P_1 P_3 \times P_1 P_2|}$
- Consideration 3: Formulate R_y as unit vector orthogonal to R_x and R_z
 - $R_y = (r_{1y} \quad r_{2y} \quad r_{3y})^T = R_z \times R_x$
- Entire transformation M is described by an orthogonal matrix and translation

$$\bullet M = \begin{pmatrix} r_{1x} & r_{2x} & r_{3x} & 0 \\ r_{1y} & r_{2y} & r_{3y} & 0 \\ r_{1z} & r_{2z} & r_{3z} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix} \cdot T(-x_1, -y_1, -z_1)$$



Example – Direction of Flight Transformation 1/3

- Example transformation task
 - Given 3D model centered in origin of x_p -, y_p -, z_p -coordinate system
 - Transform it such that
 - it is centered in arbitrary point P , and
 - the z_p -axis is directed into arbitrary direction DOF , while
 - $x_p z_p$ -plane is not rotated
- Approach: rotate first and then move
 - Rotate 3D model such that z_p -axis is oriented along DOF
 - Translate 3D model from origin into P



Example – Direction of Flight Transformation 2/3

- Step 1 – create orthogonal rotation matrix $R = \begin{pmatrix} r_{1x} & r_{2x} & r_{3x} \\ r_{1y} & r_{2y} & r_{3y} \\ r_{1z} & r_{2z} & r_{3z} \end{pmatrix}$
 - z_p -axis shall be rotated to be aligned with DOF
- $R_3 = (r_{3x} \ r_{3y} \ r_{3z})^T = \frac{DOF}{|DOF|}$
 - x_p -axis shall be rotated to be horizontal vector (i.e., orthogonal to y -axis) which is orthogonal to DOF
- $R_1 = (r_{1x} \ r_{1y} \ r_{1z})^T = \frac{y \times DOF}{|y \times DOF|}$
 - y_p -axis shall be rotated to be aligned with $R_3 \times R_1$
- $R_2 = (r_{2x} \ r_{2y} \ r_{2z})^T = \frac{DOF \times (y \times DOF)}{|DOF \times (y \times DOF)|}$

Example – Direction of Flight Transformation 3/3

- Transformation M is described by a translation and an orthogonal matrix

- $$M = T(x_p, y_p, z_p) \cdot \begin{pmatrix} r_{1x} & r_{2x} & r_{3x} & 0 \\ r_{1y} & r_{2y} & r_{3y} & 0 \\ r_{1z} & r_{2z} & r_{3z} & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}^T$$

- Degenerated case
 - If $DOF = y$, then there are infinitely many horizontal vectors
 - The cross products $y \times DOF$ and $DOF \times (y \times DOF)$ are 0 in this case



universität
uulm

3.4 Coordinate System Change

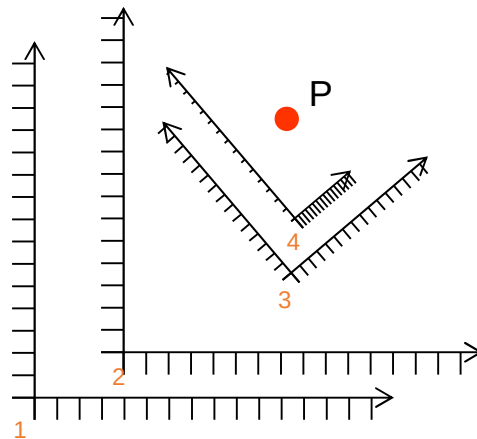
Transformation as change of frame of reference



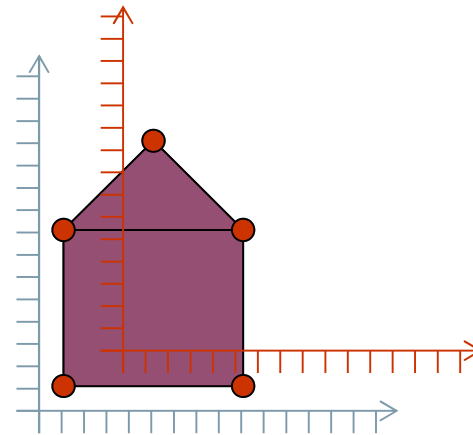
Visual Computing Group
Institute of Media Informatics

Coordinate System Change

- Alternative view on transformations:
 - transformations change the coordinate system of 3D model



Example 1



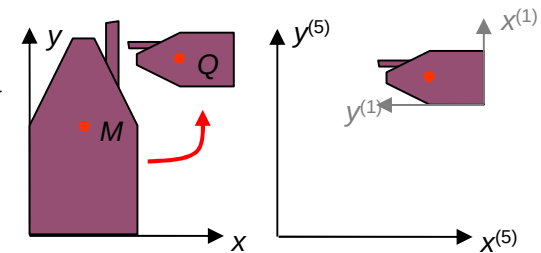
Example 2

Local Coordinate Systems

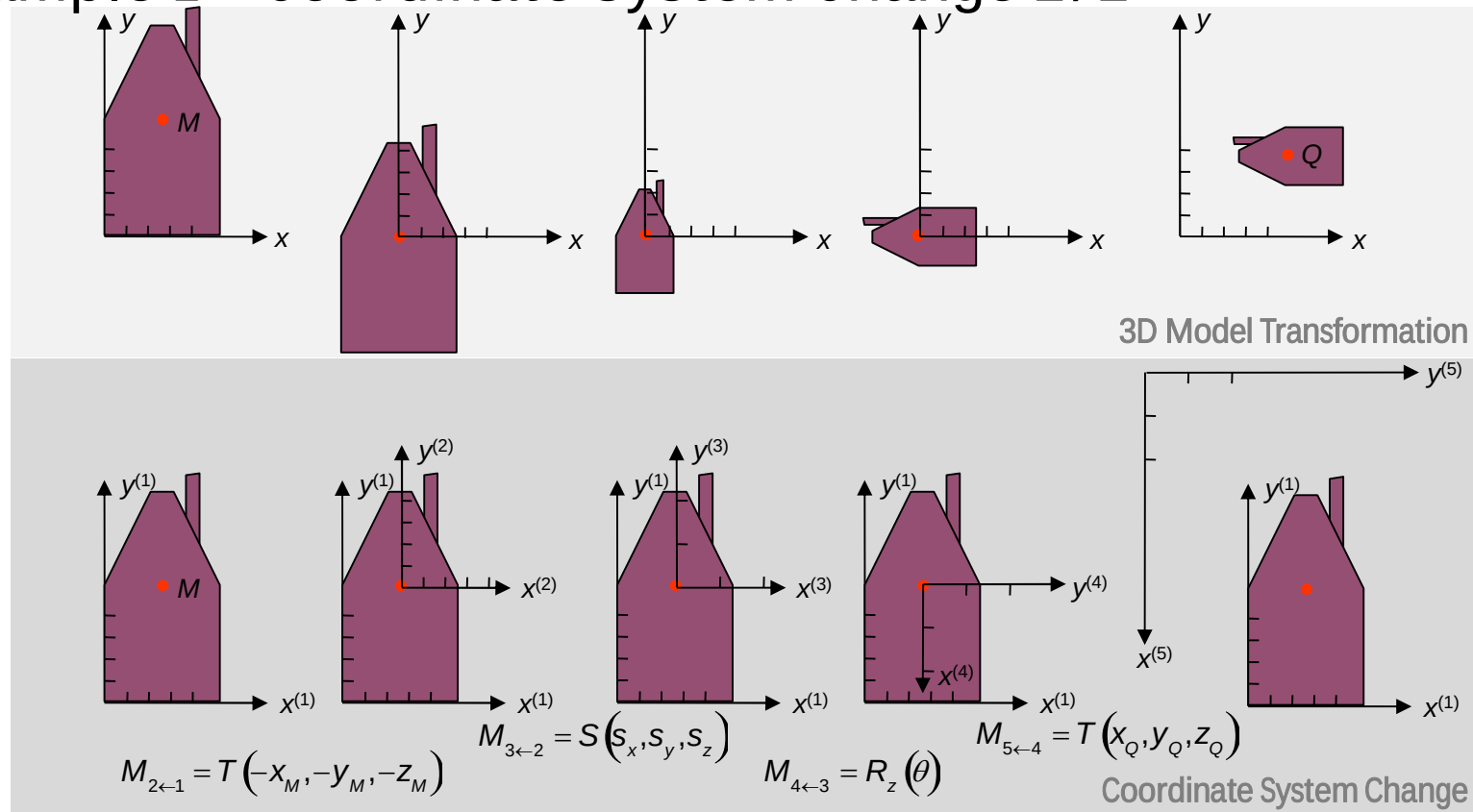
- Application in OpenGL
 - Individual 3D models of a scene have their own, local coordinate system
 - Local coordinate systems of the 3D models must be changed into a common coordinate system, the world coordinate system
- Transformation between coordinate systems (CS)
 - $M_{i \leftarrow j}$ denotes the change from CS_j to CS_i
 - $P^{(i)}$ denotes the representation of point P in CS_i
 - Inversion: $M_{i \leftarrow j} = (M_{j \leftarrow i})^{-1}$
 - Transition through multiple CSs: $M_{i \leftarrow j} \cdot M_{j \leftarrow k} = M_{i \leftarrow k}$

Example 1 – Coordinate System Change 1/2

- Example transformation task
 - Rotation and scaling of an object around its center M and subsequent translation to a point Q
- Entire transformation M is described as follows
 - $M = T(x_Q, y_Q, z_Q) \cdot R_z(\theta) \cdot S(s_x, s_y, s_z) \cdot T(-x_M, -y_M, -z_M)$
- Expressed as change of coordinate systems
 - $M_{5 \leftarrow 1} = M_{5 \leftarrow 4} \cdot M_{4 \leftarrow 3} \cdot M_{3 \leftarrow 2} \cdot M_{2 \leftarrow 1} = M$
 - $M_{1 \leftarrow 5} = M_{1 \leftarrow 2} \cdot M_{2 \leftarrow 3} \cdot M_{3 \leftarrow 4} \cdot M_{4 \leftarrow 5} = M^{-1}$
 - $M^{-1} = T(x_M, y_M, z_M) \cdot S(\frac{1}{s_x}, \frac{1}{s_y}, \frac{1}{s_z}) \cdot R_z(-\theta) \cdot T(-x_Q, -y_Q, -z_Q)$
- As applied to a point P
 - $P^{(5)} = M_{5 \leftarrow 1} \cdot P^{(1)}; P^{(1)} = M_{1 \leftarrow 5} \cdot P^{(5)}$

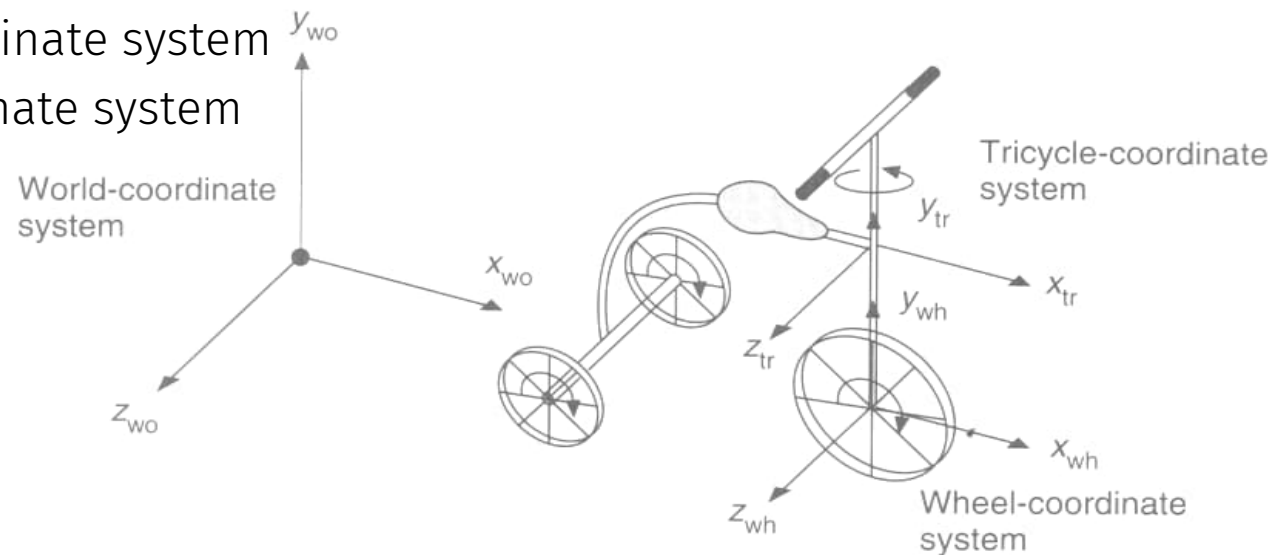


Example 1 – Coordinate System Change 2/2



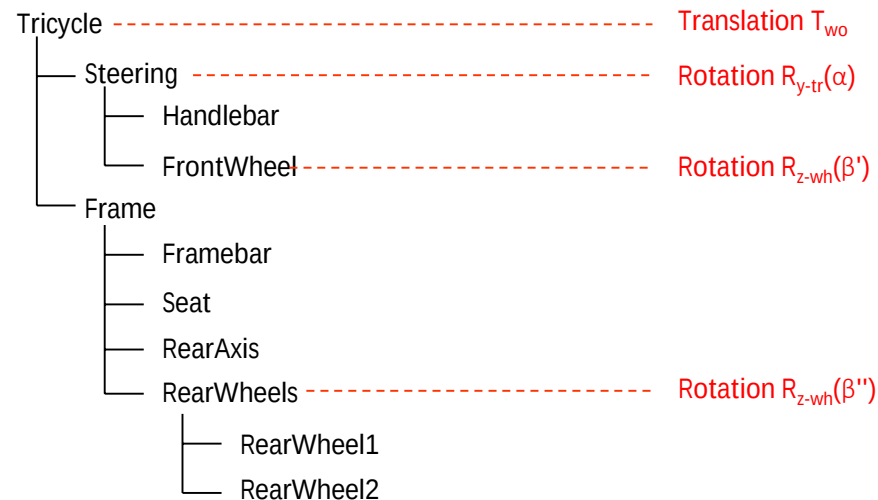
Example 2 – Hierarchical Coordinate Systems 1/2

- To model complex scenes, a hierarchy of coordinate systems is helpful
- Example: modeling of a tricycle
 - World-coordinate system
 - Tricycle-coordinate system
 - Wheel-coordinate system



Example 2 – Hierarchical Coordinate Systems 1/2

- Scene graphs are used to represent hierarchical coordinate systems and the required transformations





universität
uulm

3.5 Transformations in OpenGL

Implementation with GLM und GLSL



Visual Computing Group
Institute of Media Informatics

Geometric Transformations in OpenGL

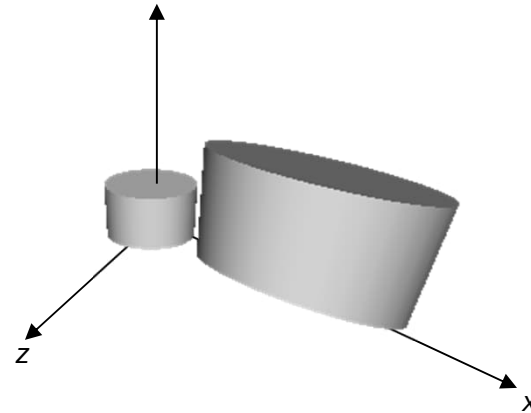
- Supported coordinate systems
 - Floating-point coordinate system for 2D and 3D
 - Integer coordinate system (2D) as a special case
- Representation of vectors
 - 2D vector by one-dimensional array with 2 elements
 - 3D vector by one-dimensional array with 3 elements
- Representation of matrices
 - 3×3 matrices by two-dimensional array with 3×3 elements
 - 4×4 matrices by two-dimensional array with 4×4 elements
- Functions for geometric transformations
 - OpenGL does not specify classes for vectors and matrices
 - GLM can be used to manage vectors and matrices

OpenGL Transformation Pipeline

- Model-View matrix
 - Transformation matrix for model space related transformations
 - Additionally contains the transformation of the scene into the camera coordinate system (view transformation, not projection!)
- Transformation pipeline
 - Each vertex is multiplied by OpenGL first with the current model-view matrix, then with the current projection matrix
 - Each standard scene should have
 - a single projection transformation
 - a single view transformation, and
 - depending on scene complexity, any number of model transformations

Example – Translation and Scaling

- Drawing cylinder objects
 - **drawCylinder** draws a cylinder centered at the origin, with respect to the y-axis, with a given radius and height
 - By translation this "unit cylinder" can be positioned
 - By scaling this "unit cylinder" can be resized



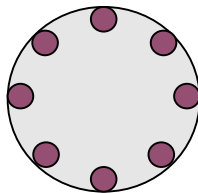
Matrix Specification

- Matrices are often specified in form of a C array

```
void draw(...) {  
    ...  
    GLfloat M[4][4];  
    M[0][0] = ....;  
    ...  
    M[3][3] = ....;  
    ...  
}
```

Example – Portico Modeling 1/5

- Columned hall consists of floor space, top surface and columns
- Columns are evenly distributed along the edge of the round floor surface
- Total height of a column is H , with a total of 20% attributable to the base
- Each column has 2 pedestals below and 2 pedestals above
- Radii
 - center column 70%
 - inner base 90%
 - outer base D

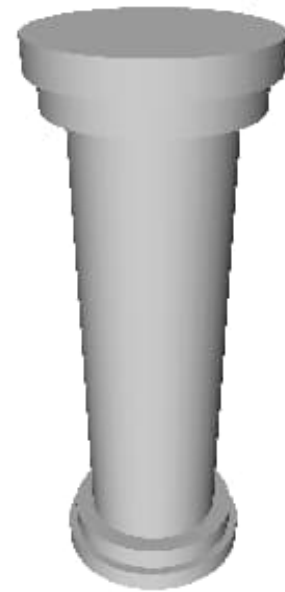


Example – Portico Modeling 2/5

- Drawing a cylinder closed at top and bottom
 - Each column should "stand" in the y-axis
 - Each column should have a lid surface at the top and bottom
- Disk primitive is embedded in the xy-plane
- Cylinder primitive consists only of lateral surface
 - Height axis equals z-axis
- Tessellation as number of polygon divisions can be specified

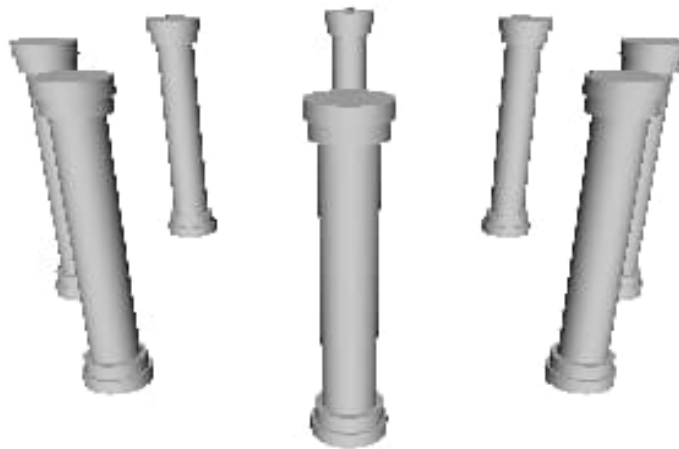
Example – Portico Modeling 3/5

- Drawing a single column
 - Composition of 5 cylinder objects
 - Rotation into y-axis to match orientation
 - Positioning of the parts by translation along the y-axis
 - Incremental translation around the partial column height



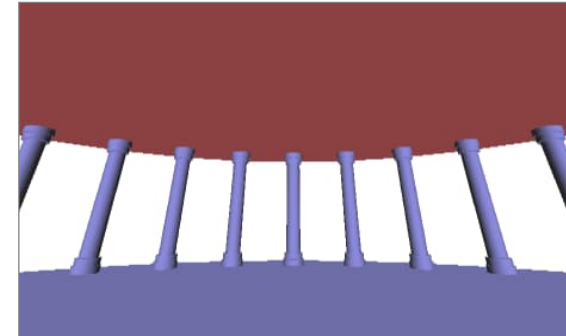
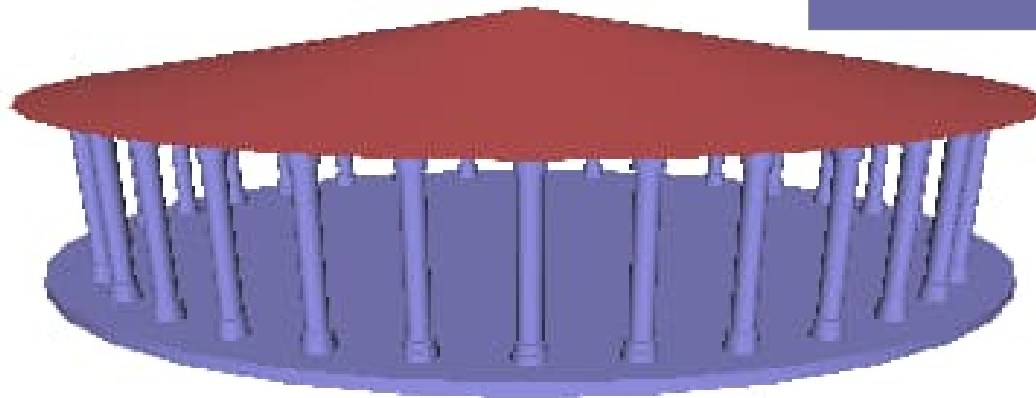
Example – Portico Modeling 4/5

- Drawing the group of columns
 - Columns are evenly distributed on a circle with radius R_H
 - Number N , radius R of the columns as well as floor radius R_H adjustable



Example – Portico Modeling 5/5

- Drawing the hall
 - Use cylinder primitive as floor area
 - Use cone primitive as a roof



Chapter Summary

- 3D models can be positioned, resized and oriented in the 3D world by applying affine transformations to their vertices
- An affine transformation is represented by a 4x4 matrix
- Affine transformations can be more efficiently applied when concatenated to a single matrix through matrix multiplication
- An alternative view on transformations is that they change the reference coordinate system



universität
uulm

References

Literature and other sources used in this chapter



Visual Computing Group
Institute of Media Informatics

References

- Text Books

- J. Foley, A. van Dam, S. Feiner: Computer Graphics: Principles and Practice (3rd Edition), Addison-Wesley 2013. (Chapters 10 & 11)
- P. Shirley, M. Ashikhmin, S. Marschner: Fundamentals of Computer Graphics (4th Edition), AK Peters 2016. (Chapter 6)